Լաբորատոր աշխատանք 5

1. Խնդրի նպատակը

Յուրացնել C++ լեզվում կառուցվածքների (struct), կառուցվածքային զանգվածների և մոդուլային ծրագրավորման (ֆունկցիաների բաժանման) գործնական կիրառումը։

2. Խնդրի պահանջը

- Սահմանել Reservoir կառուցվածք՝ name, capacity, currentVolume, dailyConsumption, region դաշտերով։
- Ստեղծել 10 ջրամբարից բաղկացած զանգված։
- Առանձին ֆունկցիաներով իրականացնել.
    1. Լցվածության տոկոսների հաշվարկ,
    2. Ամենացածր լցվածությամբ ջրամբարի որոնում,
    3. Պաշարների բավարարման օրերի հաշվարկ,
    4. Ըստ մարզի որոնում/արտածում,
    5. Տեսակավորում ըստ բավարարող օրերի աճման կարգի։

3. Տեսական մաս

- **Struct (Կառուցվածք):** Օգտագործողի կողմից սահմանվող տիպ, որը միավորում է տարբեր տիպի փոփոխականներ մեկ ընդհանուր օբյեկտի մեջ։
- **Struct-ների զանգված:** Միևնույն կառուցվածքային տիպի բազմաթիվ տարրերի հավաքածու, որոնց դաշտերին դիմում են կետ (.) օպերատորով (reservoirs[i].name):
- **Ֆունկցիաներ:** Ծրագրի տրամաբանությունը մասնատելու և կրկնվող գործողությունները մոդուլային դարձնելու միջոց, որտեղ զանգվածները փոխանցվում են հղումով կամ ցուցիչով։

```
#include <iostream>

#include <string>
```

```
#include <iomanip>
#include <algorithm>
using namespace std;
// Reservoir կառուցվածքի սահմանումը
struct Reservoir {
    string name;
    double capacity;         // Ընդհանուր տարողություն (մ³ կամ լիտր)
    double currentVolume;    // Ընթացիկ ջրի պաշար
    double dailyConsumption; // Օրական սպառում
    string region;           // Մարզ/Տարածաշրջան
};
// 1. Հաշվել և արտածել յուրաքանչյուր ջրամբարի լցվածության տոկոսը
void printFillPercentages(const Reservoir reservoirs[], int size) {
    cout << "\n--- Ջրամբարների լցվածության տոկոսները ---\n";
    for (int i = 0; i < size; ++i) {
        double percentage = 0.0;
        if (reservoirs[i].capacity > 0) {
            percentage = (reservoirs[i].currentVolume / reservoirs[i].capacity) * 100.0;
        }
        cout << fixed << setprecision(2);
        cout << reservoirs[i].name << " (" << reservoirs[i].region << "): "
            << percentage << "% (Ընթացիկ՝ " << reservoirs[i].currentVolume
            << " / Տարողություն՝ " << reservoirs[i].capacity << ")\n";
    }
}
```

```
// 2. Գտնել ամենացածր լցվածություն ունեցող ջրամբարը (տոկոսով)
Reservoir findLowestFillness(const Reservoir reservoirs[], int size) {
    int minIndex = 0;
    double minPercentage = (reservoirs[0].capacity > 0) ?
        (reservoirs[0].currentVolume / reservoirs[0].capacity) : 0.0;

    for (int i = 1; i < size; ++i) {
        double percentage = (reservoirs[i].capacity > 0) ?
            (reservoirs[i].currentVolume / reservoirs[i].capacity) : 0.0;

        if (percentage < minPercentage) {
            minPercentage = percentage;
            minIndex = i;
        }
    }
    return reservoirs[minIndex];
}
// 3. Հաշվարկել՝ քանի օր կբավականացնի ընթացիկ պաշարը յուրաքանչյուր ջրամբարի համար
void calculateDaysLeft(const Reservoir reservoirs[], int size) {
    cout << "\n--- Ջրամբարների պաշարների բավարարման օրերի քանակը ---\n";
    for (int i = 0; i < size; ++i) {
        cout << reservoirs[i].name << ": ";
        if (reservoirs[i].dailyConsumption > 0) {
            double days = reservoirs[i].currentVolume / reservoirs[i].dailyConsumption;
            cout << fixed << setprecision(1) << days << " օր\n";
        } else {
```

```
            cout << "Սպառումը 0 է (անվերջ կամ անհայտ)\n";
        }
    }
}
// 4. Արտածել տրված region-ի ջրամբարները
void printByRegion(const Reservoir reservoirs[], int size, const string& targetRegion) {
    cout << "\n--- Ջրամբարներ '" << targetRegion << "' մարզում ---\n";
    bool found = false;
    for (int i = 0; i < size; ++i) {
        if (reservoirs[i].region == targetRegion) {
            cout << "- " << reservoirs[i].name
                 << " (Տարողություն՝ " << reservoirs[i].capacity
                 << ", Պաշար՝ " << reservoirs[i].currentVolume << ")\n";
            found = true;
        }
    }
    if (!found) {
        cout << "Տվյալ մարզում ջրամբարներ չեն գտնվել:\n";
    }
}
// 5. Դասավորել ջրամբարները ըստ պաշարի բավարարող օրերի քանակի՝ աճման կարգով
void sortByDaysLeft(Reservoir reservoirs[], int size) {
    for (int i = 0; i < size - 1; ++i) {
        for (int j = 0; j < size - i - 1; ++j) {
            double days1 = (reservoirs[j].dailyConsumption > 0) ?
                (reservoirs[j].currentVolume / reservoirs[j].dailyConsumption) : 1e9;
```

```
            double days2 = (reservoirs[j + 1].dailyConsumption > 0) ?
                (reservoirs[j + 1].currentVolume / reservoirs[j + 1].dailyConsumption) : 1e9;

            if (days1 > days2) {
                // Փոխանակում ենք տեղերը
                Reservoir temp = reservoirs[j];
                reservoirs[j] = reservoirs[j + 1];
                reservoirs[j + 1] = temp;
            }
        }
    }
}
int main() {
    // Օգտագործում ենք Հայաստանի տարբեր մարզերի օրինակներով 10 ջրամբար
    const int SIZE = 10;
    Reservoir reservoirs[SIZE] = {
        {"Ազատի", 70.0, 45.0, 2.5, "Արարատ"},
        {"Մարմարիկ", 23.0, 15.0, 1.2, "Կոտայք"},
        {"Ապարանի", 91.0, 50.0, 3.0, "Արագածոտն"},
        {"Ախուրյանի", 525.0, 380.0, 12.0, "Շիրակ"},
        {"Սպիտակի", 4.5, 2.0, 0.3, "Լոռի"},
        {"Տավուշի", 5.0, 3.5, 0.4, "Տավուշ"},
        {"Ջրախբերի", 8.0, 4.0, 0.5, "Շիրակ"},
        {"Գավառագետի", 6.2, 5.0, 0.2, "Գեղարքունիք"},
        {"Կեչուտի", 23.0, 18.0, 1.5, "Կոտայք"},
        {"Սևան-Հրազդան", 35.0, 20.0, 2.0, "Կոտայք"}
```

```
    };
    // 1. Հաշվել և արտածել լցվածության տոկոսները
    printFillPercentages(reservoirs, SIZE);
    // 2. Գտնել ամենացածր լցվածություն ունեցող ջրամբարը
    Reservoir lowest = findLowestFillness(reservoirs, SIZE);
    double lowestPercentage = (lowest.capacity > 0) ? (lowest.currentVolume /
lowest.capacity) * 100 : 0;
    cout << "\n--- Ամենացածր լցվածություն ունեցող ջրամբարը ---\n";
    cout << lowest.name << " (" << lowest.region << ") - "
        << fixed << setprecision(2) << lowestPercentage << "%\n";
    // 3. Հաշվարկել քանի օր կբավականացնի պաշարը
    calculateDaysLeft(reservoirs, SIZE);
    // 4. Արտածել տրված մարզի ջրամբարները (օրինակ՝ "Շիրակ")
    printByRegion(reservoirs, SIZE, "Շիրակ");
    // 5. Դասավորել ըստ օրերի աճման կարգով և արտածել արդյունքը
    sortByDaysLeft(reservoirs, SIZE);
    cout << "\n--- Ջրամբարները՝ դասավորված ըստ պաշարի բավարարման օրերի
աճման կարգով ---\n";
    for (int i = 0; i < SIZE; ++i) {
        double days = (reservoirs[i].dailyConsumption > 0) ?
            (reservoirs[i].currentVolume / reservoirs[i].dailyConsumption) : 0;
        cout << i + 1 << ". " << reservoirs[i].name << " (" << reservoirs[i].region
            << ") - " << fixed << setprecision(1) << days << " օր\n";
    }
    return 0;
}
```